

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

Q23. A hospital system stores emergency patient data unsorted for quick updates.

Doctors need to locate records rapidly.

- a) Explain how Linear Search helps in this scenario.**
- b) Analyze its performance in different cases.**
- c) Justify why sorting may not always be preferred.**

Source Code (Python Example)

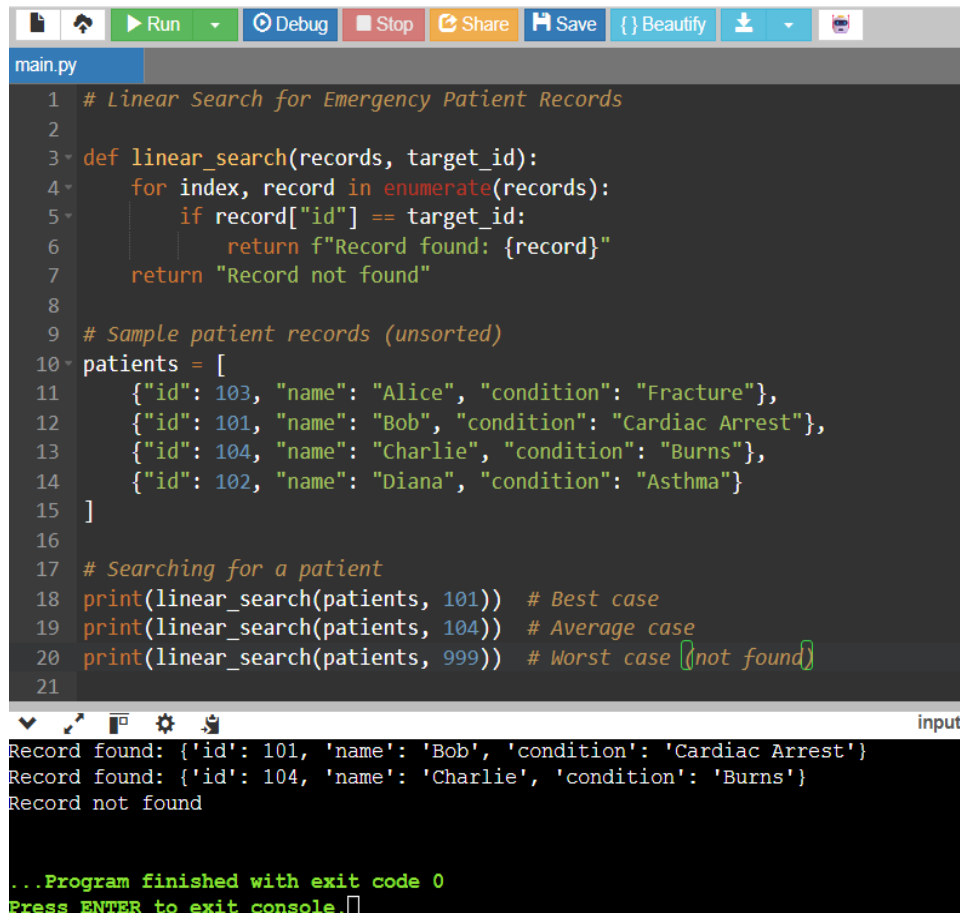
Linear Search for Emergency Patient Records

```
def linear_search(records, target_id):
    for index, record in enumerate(records):
        if record["id"] == target_id:
            return f"Record found: {record}"
    return "Record not found"

# Sample patient records (unsorted)
patients = [
    {"id": 103, "name": "Alice", "condition": "Fracture"},
    {"id": 101, "name": "Bob", "condition": "Cardiac Arrest"},
    {"id": 104, "name": "Charlie", "condition": "Burns"},
    {"id": 102, "name": "Diana", "condition": "Asthma"}
]

# Searching for a patient
print(linear_search(patients, 101)) # Best case
print(linear_search(patients, 104)) # Average case
```

```
print(linear_search(patients, 999)) # Worst case (not found)
```



```
main.py
1  # Linear Search for Emergency Patient Records
2
3  def linear_search(records, target_id):
4      for index, record in enumerate(records):
5          if record["id"] == target_id:
6              return f"Record found: {record}"
7      return "Record not found"
8
9  # Sample patient records (unsorted)
10 patients = [
11     {"id": 103, "name": "Alice", "condition": "Fracture"},
12     {"id": 101, "name": "Bob", "condition": "Cardiac Arrest"},
13     {"id": 104, "name": "Charlie", "condition": "Burns"},
14     {"id": 102, "name": "Diana", "condition": "Asthma"}
15 ]
16
17 # Searching for a patient
18 print(linear_search(patients, 101)) # Best case
19 print(linear_search(patients, 104)) # Average case
20 print(linear_search(patients, 999)) # Worst case (not found)
21

Record found: {'id': 101, 'name': 'Bob', 'condition': 'Cardiac Arrest'}
Record found: {'id': 104, 'name': 'Charlie', 'condition': 'Burns'}
Record not found

...Program finished with exit code 0
Press ENTER to exit console.
```

a) How Linear Search Helps

- **Unsorted Data Access:** In emergencies, patient records are updated frequently. Linear search allows doctors to quickly scan through unsorted data without needing preprocessing.
- **Simplicity:** It works directly on the dataset without requiring additional structures.
- **Flexibility:** Records can be inserted or updated instantly without worrying about maintaining order.

b) Performance Analysis

- **Best Case:** Patient record is the first entry → $O(1)$ time.
- **Worst Case:** Patient record is the last entry or not present → $O(n)$ time.
- **Average Case:** On average, the search will take about $n/2$ comparisons, which is still $O(n)$.

c) Why Sorting May Not Always Be Preferred

- **Time Overhead:** Sorting after every update is costly ($O(n \log n)$).
- **Dynamic Updates:** Emergency records change rapidly; maintaining sorted order slows down insertion.
- **Real-Time Needs:** Doctors prioritize immediate access over optimized search.

Case	Description	Time Complexity
Best	Record is the first entry	$O(1)$
Average	Record is somewhere in the middle	$O(n/2) \approx O(n)$
Worst	Record is last or not present	$O(n)$

Linear Search is a practical solution for hospital emergency systems where:

- Data is unsorted due to frequent updates.
- Doctors require immediate access to patient records.
- Sorting introduces unnecessary delays.

While not the most efficient algorithm for large datasets, Linear Search balances **speed, flexibility, and simplicity**, making it suitable for real-time healthcare applications.